دفتر 20 / من الشرح للتطبيق

نبرمج لعبة تخمين الرقم

الكمبيوتر يختار رقما من 1 إلى 20، وإنت معاك خمس محاولات صحيحة للوصول له. بعد كل تخمين هيقولك تطلع ولا تنزل. هنكتب القواعد الأول علشان العداد والسلوك يبقوا واضحين.

في آخر الدفتر هتقدر تعمل إيه؟

تشغل لعبة لها فوز وخسارة وخروج، وتتحقق من الإدخال، وتختبرها برقم ثابت قبل تشغيل الاختيار العشوائي.

هنمشي بالترتيب ده

1. نحدد قواعد المحاولات والإدخال والخروج.
2. نفصل مقارنة التخمين عن حلقة اللعبة.
3. نختبر الفوز والخسارة والإدخال غير الصالح.

قبل ما تبدأ

while وcontinue والدوال والاستثناءات. اكتب جزأي الصفحتين 2 و3 في ملف guessing_game.py واحد، والمقارنة قبل اللعبة.

افتح ملف كود جنب الدفتر. اقرأ المثال، توقع النتيجة، وبعدها شغله. لو النتيجة مختلفة، ارجع للسطر اللي غيرها بدل ما تعيد نسخ الكود كله.

خريطة الدفتر: الفكرة ص 2 • تطبيق خطوة بخطوة ص 3 • الأخطاء ص 4 • تمارين ص 5 • حلول ص 6 • مراجعة ص 7

بايثون مع نوار

المشروع الثاني

شرح • أمثلة • تمارين • حلول

20

01 / القواعد والجزء الأول

كل محاولة لها معنى

المجال من 1 إلى 20 شامل الطرفين. العدد الصحيح داخل المجال يستهلك محاولة حتى لو كان مكررا. النص غير الرقمي أو العدد خارج المجال لا يستهلك محاولة. الحرف q ينهي اللعبة. اللاعب يفوز عند التطابق ويخسر بعد خمس محاولات صحيحة غير مطابقة.

```python
def compare_guess(guess, secret):
    if guess < secret:
        return "Higher"
    if guess > secret:
        return "Lower"
    return "Correct"
```

الدالة لا تقرأ من المستخدم ولا تغير العداد. هي فقط تقارن وترجع رسالة: Higher تعني أن الرقم السري أكبر من التخمين، وLower تعني أنه أصغر، وCorrect تعني التطابق.

ليه نمرر الرقم السري؟

play ستستقبل secret كمدخل. أثناء الاختبار نمرر 10 مثلا لنعرف النتيجة مسبقا. وقت اللعب الحقيقي نولد الرقم بـ random.randint. الاختيار هنا للعبة، وليس لأغراض أمنية.

تحقق سريع

compare_guess(5, 10) ترجع Higher، وcompare_guess(15, 10) ترجع Lower. جرب الحالتين قبل حلقة اللعبة.

02 / الجزء الثاني

حلقة اللعبة كاملة

PYTHON

```python
import random

def play(secret, max_attempts=5):
    attempts = 0
    while attempts < max_attempts:
        raw = input("Guess (1-20), or q: ").strip()
        if raw.lower() == "q":
            print("Bye")
            return
        try:
            guess = int(raw)
        except ValueError:
            print("Use a whole number")
            continue
        if not 1 <= guess <= 20:
            print("Stay between 1 and 20")
            continue
        attempts += 1
        message = compare_guess(guess, secret)
        print(message)
        if message == "Correct":
            print(f"Attempts: {attempts}")
            return
        print(f"Remaining: {max_attempts - attempts}")
    print(f"No attempts left. Secret: {secret}")

if __name__ == "__main__":
    play(random.randint(1, 20))
```

03 / نفك الحلقة

العداد يزيد بعد التحقق

1. الإدخال والخروج

strip تزيل مسافات الطرفين. نفحص q قبل التحويل لرقم، ونستخدم lower لقبول Q أيضا. return تنهي الدالة، لذلك لا تظهر رسالة الخسارة بعد خروج اللاعب أو فوزه.

2. التحويل والمجال

int ترفض النص غير الصحيح. except تعرض رسالة ثم continue تعيد الطلب بدون زيادة المحاولات. نفحص المجال بعدها؛ لو العدد خارج المجال نكمل للدورة التالية أيضا بدون زيادة.

3. مقارنة ومحاولة محسوبة

بعد التحقق، نزيد attempts مرة واحدة. نقارن التخمين، ولو كان صحيحا نعرض عدد المحاولات ونرجع. غير كده نعرض المتبقي. عند استهلاك الحد كله تنتهي while وتظهر رسالة الخسارة والرقم السري.

4. الاختبار قبل العشوائية

استبدل استدعاء النهاية مؤقتا بـ play(10). جرب 5 ثم 15 ثم 10: النتيجة Higher ثم Lower ثم Correct، وعدد المحاولات 3. جرب hello ثم 10؛ الفوز يجب أن يكون في محاولة واحدة.

حالات لا تنساها

جرب 0 و21، وحرف q، وخمس تخمينات خاطئة داخل المجال. جرب الفوز في المحاولة الخامسة؛ يجب ألا تظهر رسالة الخسارة بعده.

جرب قبل ما تبص على الحل

دورك إنت

تطبيق 01 / تتبع اللعب

الرقم السري 10. الإدخال: hello ثم 0 ثم 5 ثم 10. كم محاولة عند الفوز؟

تلميح صغير

العداد يزيد بعد تحقق النوع والمجال.

تطبيق 02 / عدد محاولات مختلف

شغل اللعبة بسبع محاولات وبرقم سري ثابت أثناء الاختبار.

تلميح صغير

max_attempts له قيمة افتراضية تقدر تغيرها عند الاستدعاء.

تطبيق 03 / راجع مكان العداد

إيه اللي يحصل لو زودنا attempts قبل int بدل بعدها؟

تلميح صغير

اسأل هل الإدخال الخاطئ وصل لسطر الزيادة.

قبل صفحة الحل: جرب حالة عادية وحالة على الحدود. اكتب النتيجة المتوقعة الأول، وبعدها قارنها باللي طلع فعلا.

مساحة لتجربتك

حلول مشروحة / قارن طريقة التفكير

نراجع اللي عملناه

حل 01 / تتبع اللعب

محاولتان. hello و0 لا يستهلكان محاولات، و5 و10 محاولتان صحيحتان من ناحية الإدخال.

حل 02 / عدد محاولات مختلف

PYTHON

```
play(10, max_attempts=7)
```

بعد تعريف الدوال، استخدم الاستدعاء التالي. لا تحتاج تغيير جسم الحلقة.

حل 03 / راجع مكان العداد

وقتها الإدخال غير الرقمي هيستهلك محاولة، وده يخالف القاعدة التي اتفقنا عليها. مكان السطر جزء من تصميم اللعبة.

ورقة ترجع لها بسرعة

قبل ما تقفل الدفتر

راجع نفسك

- جمعت المقارنة واللعبة في ملف واحد.
- اختبرت برقم ثابت ثم رجعت العشوائية.
- فرقت بين الخروج والفوز والخسارة.
- راجعت متى يزيد العداد.

المصطلحات من غير تعقيد

State : القيم التي تصف حالة اللعبة الآن.

Random : اختيار شبه عشوائي مناسب للعبة.

Boundary case : حالة على طرف المجال أو حد المحاولات.

اختبار الفهم

ليه return بعد الفوز مهمة؟ لأنها تمنع استمرار الحلقة والوصول لرسالة الخسارة. break تحتاج تنظيما إضافيا لتمييز الفوز بعد الخروج.

نكمل على إيه؟

كده خلصت موضوعات الموقع الحالية. طور المشروع بخيار إعادة اللعب أو حفظ أفضل نتيجة، ثم اختبر كل ميزة لوحدها.

لما تحتاج التفاصيل الرسمية

الشرح والأمثلة هنا مكتوبة للمبتدئ. الروابط التالية مراجع للمراجعة والتوسع، وليست نصا منقولا منها.

المكتبة القياسية random

https://docs.python.org/3/library/random.html

الحلقات والدوال

https://docs.python.org/3/tutorial/controlflow.html